

Звіт з практичної роботи № 3

Управління життєвим циклом програмного продукту та інструменти програмної інженерії

Виконавець: Озійчук Ярослав Дата: 16.09.2026

Дана робота присвячена дослідженню та практичному застосуванню методологій управління життєвим циклом розробки програмного забезпечення (SDLC) на прикладі проєкту веб-системи. У ході роботи виконується вибір моделі розробки, планування фаз, створення дорожньої карти та налаштування інструментів ALM для ефективної командної взаємодії.

Завдання 1. Вибір моделі SDLC

Картка проєкту

- **Назва проєкту:** Веб-система електронного документообігу та обліку задач "TaskFlow".
- **Тип проєкту:** Web-орієнтований додаток.
- **Замовник:** Внутрішній корпоративний замовник (ІТ-відділ компанії).
- **Розмір команди:** 4 особи.
- **Очікуваний термін:** 3 місяці.
- **Стабільність вимог:** Вимоги можуть змінюватись залежно від відгуків користувачів.

Оцінка проєкту за критеріями

Традиційна (Waterfall)
Чітко визначений, великий
Стабільні, незмінні

Критерій оцінки Гнучка (Agile/Scrum/Kanban)

Обсяг та складність Невизначений, змінюваний

Вимоги Гнучкі, еволюційні

Традиційна (Waterfall)
Ієрархічна, формальна
Залучений на старті та фініші
Жорстко фіксовані

Критерій оцінки Гнучка (Agile/Scrum/Kanban)

Команда та взаємодія Самоорганізована, тісна

Замовник Постійно залучений в ітерації

Терміни та бюджет Гнучкі, орієнтовані на цінність

Орієнтир для проєкту Обрано: Agile (Scrum)

Обґрунтування вибору Agile (Scrum)

Вибір методології Agile (Scrum) для проєкту "TaskFlow" зумовлений потребою в максимальній гнучкості та швидкості адаптації до вимог внутрішнього замовника. Враховуючи невеликий розмір команди (4 особи) та стислі терміни розробки, Scrum дозволяє ефективно розподіляти ресурси через короткі ітерації (спринти), забезпечуючи постійну поставку робочого функціоналу. Такий підхід суттєво зменшує ризики розробки непотрібних функцій та дозволяє коригувати пріоритети без зупинки всього процесу. Постійна взаємодія з

замовником під час Sprint Reviews гарантує, що фінальний продукт максимально відповідатиме реальним потребам корпоративного середовища. Ітераційне тестування та інтеграція сприяють високій якості коду вже на ранніх етапах.

Завдання 2. Фази життєвого циклу та дорожня карта

Таблиця фаз життєвого циклу (SDLC)

Зміст робіт
Збір та уточнення вимог, опис User Stories

Фаза ЖЦ **Тривалість** Аналіз вимог2 тижні

Зміст робіт
Проектування архітектури, БД та API
Кодування фронтенд та бекенд частин
Функціональне та інтеграційне тестування
Налаштування серверів та запуск у прод

Фаза ЖЦ **Тривалість** Проектування2 тижні

Розробка5 тижнів Тестування2 тижні Розгортання1 тиждень

Дорожня карта (Roadmap)

- **Тижні 1-2:** Аналіз бізнес-процесів, затвердження ТЗ та прототипів інтерфейсу.
- **Тижні 3-4:** Розробка архітектури БД, налаштування середовища розробки та CI/CD.
- **Тижні 5-6 (Спринт 1):** Базовий функціонал авторизації та створення структури задач.
- **Тижні 7-8 (Спринт 2):** Реалізація системи статусів, фільтрації та документообігу.
- **Тижні 9-10 (Спринт 3):** Система сповіщень, інтеграція з календарем та баг-фіксинг.
- **Тиждень 11:** Повне тестування (UAT) та документація для користувачів.
- **Тиждень 12:** Фінальний реліз, демонстрація замовнику та перехід у режим підтримки.

Завдання 3. Дошка задач (ALM-інструмент)

Для управління проектом обрано інструмент **GitHub Projects** за методологією Kanban. Дошка налаштована для забезпечення прозорості потоку задач та швидкої ідентифікації "вузьких місць".

Налаштування дошки (Колонки)

1. **Backlog:** Усі ідеї та майбутні завдання.
2. **To Do:** Задачі, обрані для поточного спринту.
3. **In Progress:** Задачі, над якими зараз ведеться робота.
4. **Review:** Задачі, що очікують на перевірку коду (Pull Request).
5. **Done:** Повністю завершені та протестовані завдання.

Перелік задач проекту

1. [Feature] Розробка схеми бази даних (Epic: Core Architecture).
2. [DevOps] Налаштування GitHub Actions для автоматичного

тестування.

3. [Feature] Реалізація JWT-авторизації користувачів.
4. [Feature] Створення інтерфейсу списку задач (Dashboard).
5. [Feature] Модуль завантаження та зберігання документів.
6. [Documentation] Створення OpenAPI специфікації для API.
7. [Bug] Виправлення помилки відображення статусів на мобільних пристроях.
8. [Feature] Реалізація коментарів до задач.
9. [Feature] Пошук та фільтрація документів за тегами.
10. [DevOps] Розгортання Docker-контейнерів на сервері.
11. [Documentation] Інструкція адміністратора системи.
12. [Feature] Генерація щотижневих звітів у PDF.

Задачі розміщуються на візуальній дошці у відповідних колонках згідно з їх поточним статусом. Наприклад, задачі 1 та 2 знаходяться в "Done", 3-5 у "Review", а решта розподілена між "In Progress" та "To Do".

Завдання 4. Sprint Planning

Ціль спринту (Sprint Goal): Створити стабільну основу системи з можливістю реєстрації користувачів та базового управління життєвим циклом задачі.

Таблиця Sprint Plan

Конкретні дії
Створення репозиторію та ініціалізація проєкту

ID Оцінка (год) T-0014

Конкретні дії
Розробка API-ендпоінтів для реєстрації та входу

Верстка головної сторінки Dashboard
Логіка створення та редагування задач (CRUD)
Покриття Unit-тестами основних сервісів

ID Оцінка (год) T-00212

T-00310 T-00416 T-0058

Definition of Done (DoD)

- 1. Код пройшов Code Review та відповідає стандартам стилю.
- 2. Усі Unit-тести виконані успішно (Coverage > 80%).
- 3. Документація API оновлена у Swagger/OpenAPI.
- 4. Функціонал перевірено на відповідність критеріям прийнятності (Acceptance Criteria).
- 5. Гілка змерджена в main та успішно розгорнута на стейджинг сервері.
- 6. QA підтвердив відсутність критичних дефектів у новій фічі.

Завдання 5. DORA-рефлексія

Аналіз ефективності команди за метриками DORA:

- 1. **Deployment Frequency:** Використання Kanban та CI/CD дозволяє проводити деплой декілька разів на тиждень, що наближає команду до рівня *High/Elite*.
- 2. **Lead Time for Changes:** Завдяки декомпозиції задач на дрібні підзадачі на дошці GitHub Projects, час від комміту до релізу скорочується до декількох годин.
- 3. **Change Failure Rate:** Обов'язковість Unit-тестів та Review (згідно з

DoD) мінімізує ризик збоїв при релізах, тримаючи показник нижче 5%.

4. **MTTR (Mean Time to Recovery):** Автоматизований моніторинг та швидкий відкат змін через GitHub дозволяють відновлювати систему менш ніж за годину.

Контрольні запитання

1. **Що таке життєвий цикл розробки ПЗ (SDLC) і які основні фази він включає?** SDLC — це структура, що визначає етапи розробки програмного продукту від ідеї до виведення з експлуатації. Основні фази: аналіз вимог, проєктування архітектури, розробка (кодування), тестування, розгортання та супровід.
2. Чим відрізняється каскадна модель (Waterfall) від гнучких методологій (Agile)? Waterfall передбачає послідовне виконання фаз, де перехід до наступної можливий лише після завершення попередньої. Agile — ітеративний підхід, де продукт розробляється короткими циклами, що дозволяє вносити зміни на будь-якому етапі.
3. У чому полягає основна різниця між Scrum та Kanban? Scrum базується на фіксованих ітераціях (спринтах) з чіткими ролями (Scrum Master, Product Owner). Kanban фокусується на безперервному потоці задач та обмеженні кількості одночасної роботи (WIP limits) без жорстких часових рамок спринтів.
4. Яка роль систем контролю версій (VCS) в управлінні життєвим циклом ПЗ? VCS (наприклад, Git) дозволяє зберігати історію змін, забезпечує паралельну роботу декількох розробників над одним проєктом, дозволяє відкочуватись до попередніх версій та є основою для процесів CI/CD.
5. **Що вимірюють метрики DORA і чому вони важливі для сучасної програмної інженерії?** Метрики DORA (частота розгортання, час виконання змін, частка збоїв, час відновлення) вимірюють продуктивність та стабільність команди розробки. Вони допомагають виявити слабкі місця в процесах та об'єктивно оцінити ефективність DevOps-практик.
6. **Як інструменти автоматизації CI/CD впливають на швидкість та якість розробки?** CI/CD автоматизує тестування та доставку коду, що зменшує кількість ручних помилок, пришвидшує отримання фідбеку та дозволяє

частіше випускати оновлення без втрати стабільності системи.

7. Які переваги надає використання ALM-інструментів (Jira, GitHub Projects, Trello)? ALM-інструменти забезпечують централізоване управління задачами, візуалізацію робочого процесу, прозорість відповідальності, відстеження прогресу та ефективну комунікацію між усіма учасниками проєкту.